Vidya Balachander, Abir Lal
Student IDs: 2432794, 2470634
CSE/EE 371
November 01, 2026
Lab Report 3: Digital Signal Processing

Design Procedure

In this lab, we were tasked with using the audio coder/decoder on the DE1_SoC board to generate and filter noise from both an external source and internal memory. We also created an FIR (Finite Impulse Response) filter, which used moving averages of the sample data to reduce audio noise and improve perceived audio quality. We tested and tweaked the amount of averaging in our filter to find the perfect value to balance audio quality, audio volume, and memory utilization.

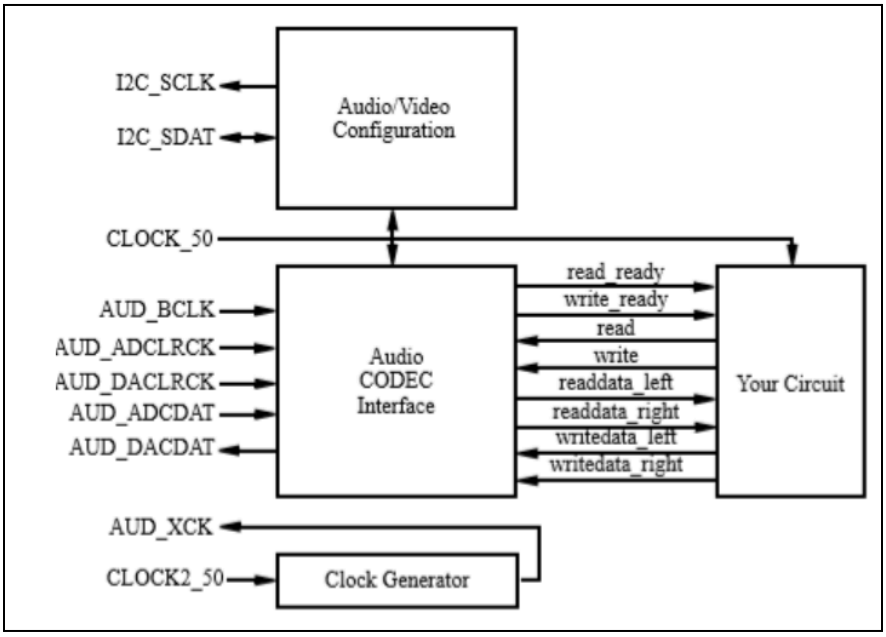


Figure 1: Provided block diagram of the audio system we used in this lab.

Task #1

In this task, we played a music file into the audio input of the DE1-SoC, which we simultaneously played through the audio output of the device. We were provided with some starter code files for this task, which consisted of a Clock Generator, an Audio CODEC Interface, and an Audio/Video Configuration module. Our first task was to edit the provided starter files to pass the audio input to audio output only when the CODEC is ready. After reading through the spec, we realized that the CODEC is only ready when both the one-bit read_ready and write_ready signals are true. We set both our read and write signals to be equal to read_ready & write_ready. We also set write_data_left to equal read_data_left and write_data_right to equal

read_data_right. After checking our logic with our TA, we tested our code on LabsLand by uploading the piano_noisy.mp3 file to the program. We played the audio into the board and recorded the output at the same time, which was a match.

Task #2

In Task 2, we played a tone using on-chip ROM memory. We first created the tone, for which we used a Python script to generate a 1-second .mif file of the arbitrarily chosen music note D5. We then used the Quartus library to create and initialize the On-board ROM, using the .mif file data as the ROM's memory content data. To play this audio, we needed to output each value stored in the ROM sequentially to the AUDIO codec, and then use the speaker to play those values, looping back to the start when we ended. To do this, we used a helper module with an always_ff block to output the value stored at the current ROM address on the clock edge if the write signal was enabled, which depended on write_ready. Then, we would increment to the next ROM address while simultaneously incrementing a counter variable. We used a counter to ensure the data looped from the last address in the ROM that stored a valid sample back to the beginning. This theoretically creates an audio loop that would keep playing. Finally, in our new top-level module (task2.sv), we used ternary logic to select between outputting data based on recorded audio (Task 1) or the on-board ROM (Task 2) based on the value of SW9. We then recorded the output produced by the board, which was indeed a loop of the D5 tone.

Task #3

In this task, we created an FIR (Finite Impulse Response) filter to remove noise from our audio by averaging the values of N adjacent samples. To do this, we created a FIFO buffer, implemented using the circular queue created in homework #3. The specification provided for this lab suggested that we could use different values of N. After testing a few different values, we settled on a value of 512 based on perceived noise reduction, audio volume, as well as memory optimization, to reduce the size of the FIFO buffer. This buffer stores the previous N 24-bit data values of the audio, all divided by N. We used a 9-bit shifting method for division, which was the reason for selecting a value of N that was an exponent of 2. As the values are input, they are first divided by N, then added to the total value, which is stored in an accumulator register. This 24-bit value is then output to the Audio Codec. When the next value is input, it is once again divided by N and added to the total. It then enters the buffer, from which the oldest entry is subsequently removed. This removed value is then subtracted from the total, which now has a new value to output to the Audio Codec. After the FIFO is full, these two operations will always occur before the next value is pushed to the Codec. Before the FIFO fills up, i.e for the first N-1 values, we simply output the sum of the 9-bit shifted values that were input so far. We judged that since this is a negligible timeframe to the ear (roughly 1/100th of a second), this is an acceptable trade-off. This system essentially stores the average of N consecutive inputs, which is the output value. When the next value is input, a new average is calculated and output. We created yet another new top-level module for this task, which we named DE1_SoC.sv.

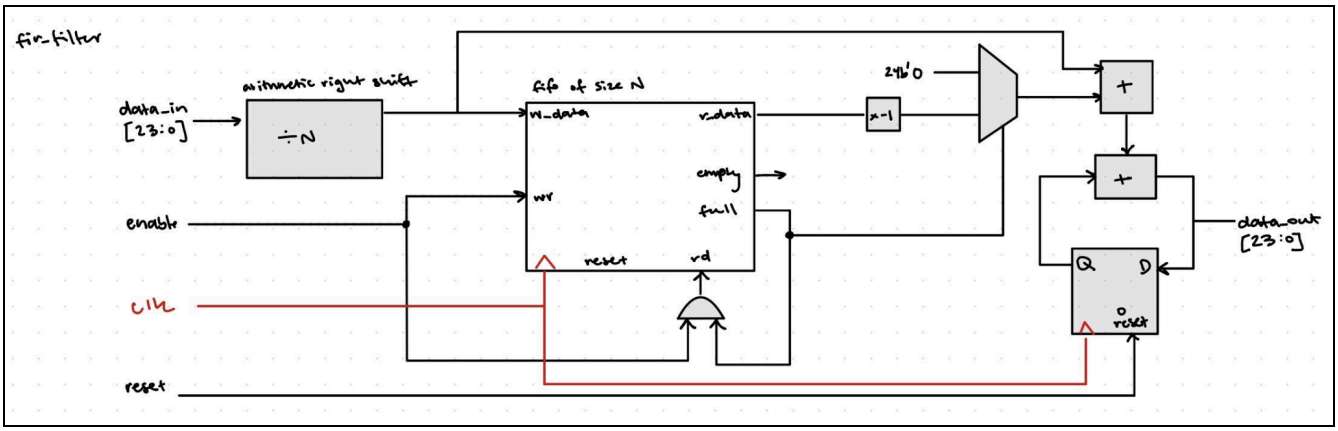


Figure 2: Implementation of the N-sample averaging FIR that we used in this lab.

Overall System

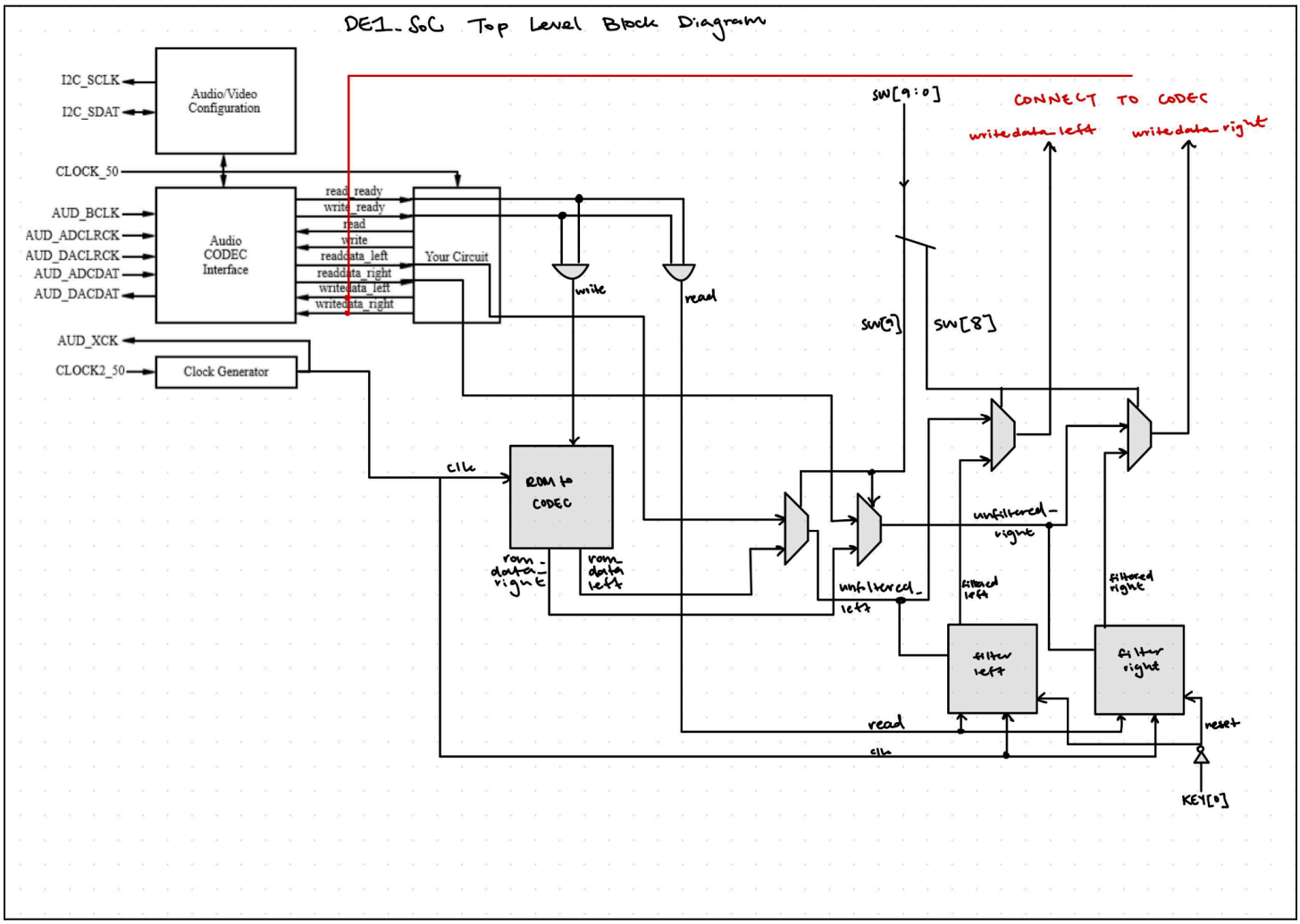


Figure 3: The top-level block diagram for the DE1_SoC

The overall block diagram for this lab report is shown above. Note that we weren't able to show the full filter implementation on this diagram. The 'filter left' and 'filter right' blocks, which are used respectively to filter the left and right channels, take the place of the fir_filter implementation shown above the overall system block diagram.

Results

Our system essentially processes two 24-bit channel audios on the DE1_SoC board. Using a switch(SW9) on the board, the system selects whether to play back audio it records in real time or to play audio from the on-board ROM. A separate switch (SW8) selects whether or not to filter the audio using a 512-sample moving-average FIR filter. Separate FIFO buffers filter the left and right channels, feeding filtered data into the audio CODEC. The on-board Key-0 handles the reset logic for the system.

`rom_to_codec_tb`

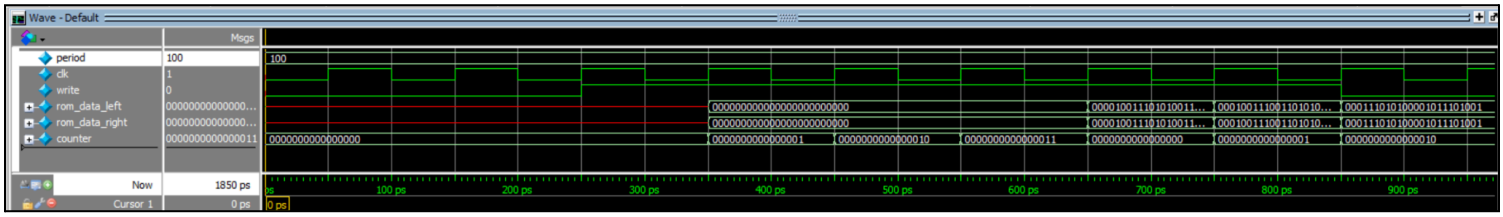


Figure 4: The first half of the ModelSim waveform for the rom_to_codec testbench module

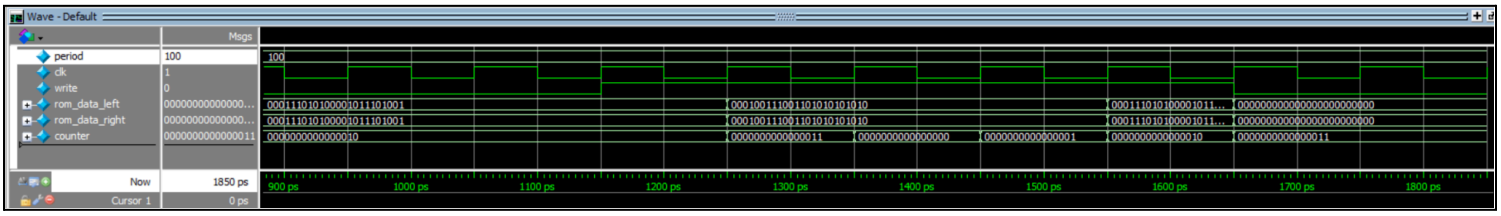


Figure 5: The second half of the ModelSim waveform for the task1 testbench module

This testbench tested out the helper module that we used to feed values from where they were stored in the ROM to the audio codec. To do this, we used a counter variable that we incremented each iteration of the always_ff block. When the counter reaches a defined parameter counter_max, it wraps around to a counter of 0. Using the counter as the address of the ROM that we access in the current iteration, we are able to sequentially output values to the codec. In the testbench, we tested whether counter correctly increments and correctly wraps around to 0 after reaching counter_max. For the sake of simplicity, we set counter_max to 4. We also tested whether writing was paused when the write enable was off, and whether values held (the counter stayed at the current address) when the write enable was off. As expected, the counter wrapped back to 0 after reaching 011. The testbench also passed the other described tests successfully, as expected.

`fir_filter_tb`

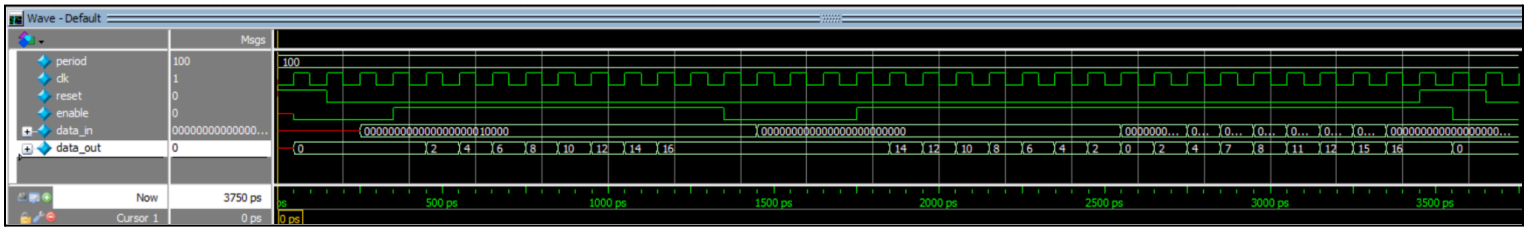


Figure 6: The ModelSim waveform for the fir_filter testbench module

For the fir_filter module, there were a few basic functionalities to test. Firstly, we used an 8-sample averaging FIR filter, for simplicity and convenience. This allowed us to rapidly test functionality such as pushing values out of the FIFO buffer. In this testbench, we first filled the buffer with 8 identical samples, which in turn resulted in equal incrementation of the output value. We then tested whether replacing the old values with new, identical values, would have the output stay the same - which it did. We tested whether the buffer truly was dependant on the enable signal by turning it off and then attempting to write. We also tried to write 0 values, which resulted in the sequential decrementing of the output value. We tested whether the buffer supported inputting different values. Finally, we also tested whether the reset functionality was active and whether the accumulator would return an output of 0 after being reset. The testbench passed all these tests successfully.

Flow Summary

Flow Summary

<<Filter>>

Flow Status	Successful - Sun Nov 01 23:24:31 2026
Quartus Prime Version	17.0.0 Build 595 04/25/2017 SJ Lite Edition
Revision Name	DE1_SoC
Top-level Entity Name	DE1_SoC
Family	Cyclone V
Device	5CSEMA5F31C6
Timing Models	Final
Logic utilization (in ALMs)	411 / 32,070 (1 %)
Total registers	491
Total pins	21 / 457 (5 %)
Total virtual pins	0
Total block memory bits	1,188,864 / 4,065,280 (29 %)
Total DSP Blocks	0 / 87 (0 %)
Total HSSI RX PCSs	0
Total HSSI PMA RX Deserializers	0
Total HSSI TX PCSs	0
Total HSSI PMA TX Serializers	0
Total PLLs	1 / 6 (17 %)
Total DLLs	0 / 4 (0 %)

Figure 10: The flow summary for the top-level DE1_SoC module

Experience Report

This lab was not very difficult and quite engaging. A lot of the necessary materials were already provided to us, such as the Audio Codec and the circular queue-based FIFO implementation. The interactiveness of the audio was also a plus point for this lab. That being said, it did take us some time to understand the concepts introduced in the lab, particularly the FIFO buffer. Writing testbenches was tough because we didn't exactly know what to test for, since most of the functionality was provided in the Audio Codec, which we didn't write. For the most part, we were able to use LabsLand, but we would have definitely preferred using a physical board in this lab. We weren't able to procure the speaker peripheral, so this wasn't possible.

Once again, it was very useful to break the lab down into discrete steps. This helped us know which task and deliverable exactly was next on the list. Also, writing this report as we were writing the code was helpful for keeping track of our tasks. For testing our N-values, we initially used trial and error, synthesizing and uploading our code to LabsLand multiple times. We then realized that we could simply use switches to change the N-value during just one synthesis. This saved a lot of time during testing.

Overall, this lab took us around 12.5 hours, broken down as follows:

- Reading/Planning - 1 hour
- Coding - 4 hours
- Testing - 2.5 hours
- Debugging - 2 hours
- Lab report - 1 hour
- Miscellaneous (comments, labsland, diagrams) - 2 hours